Common Core: Phase 3 Physical Design

Lab P2: Static Timing Analysis

Constraints, Timing Reports, Critical Paths and Fmax

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 18, 2026

Lab objective: Constrain the netlists from Lab P1 with an SDC file, analyze them with OpenSTA, read every line of a timing report, connect the critical path to your RTL, and use the results to compare architectures and improve one.

Duration	1.5 weeks (approximately 25–30 hours)
Prerequisites	Lab P1
Designs	`seq_mul`, `comb_mul` (R1), `sync_fifo` (R2); one RTL improvement to `seq_mul`
Reused in	P3 (sweeps), T-RTL1 and T-RTL2 (multi-clock SDC), T-RTL3 (frequency target)

Contents

1 Overview **3**

2 Static Timing Analysis Concepts **4**

2.1 Paths, Startpoints and Endpoints . . . 4

2.2 Setup and Hold . . . 4

2.3 Recovery and Removal . . . 5

2.4 What This Analysis Leaves Out . . . 5

2.5 Timing Exceptions . . . 6

3 Constraints: the SDC File **6**

4 The Generic STA Script and `sta.mk` **7**

5 Anatomy of a Timing Report **8**

5.1 The Columns . . . 9

5.2 The Annotations . . . 9

5.3 A Hold Report . . . 10

5.4 The Same Report in PrimeTime . . . 10

6 Timing Analysis of `seq_mul` and `comb_mul` **11**

7 Improvement Task **12**

8 `sync_fifo`: Effect of WIDTH and DEPTH **13**

9 Requirements / Tasks **13**

10 Repository and Flow **14**

11 Deliverables **14**

12 Assessment **15**

13 Review Questions **15**

14 References **16**

1 Overview

Every chip is designed for a clock frequency, and the question "does it run at that frequency?" is not answered by simulation. A simulation checks the paths that the test happens to exercise, with zero delay; a real design has millions of paths, and the slowest one decides the frequency. **Static timing analysis** (STA) answers the question exhaustively and without stimulus: it computes, for every path between flip-flops and ports, the latest and earliest time a signal can arrive and compares it with the time the clock requires.

In a real team, STA is run by RTL designers (to check the micro-architecture early), by the PD team after every implementation step, and finally for sign-off at all corners. Its input is not only the netlist but also the **constraints** (an SDC file): what the clock is, when inputs arrive, and what is connected to the outputs. Wrong constraints produce a correct-looking report about the wrong problem, so writing and reviewing them is a core skill.

In this lab you write SDC files for your blocks, a generic OpenSTA script, read an annotated timing report line by line, answer the timing questions for `seq_mul`, compare it with `comb_mul`, improve the critical path of `seq_mul`, and study how the FIFO parameters change its timing.

On the Synopsys cloud: STA runs in **PrimeTime** (`pt_shell`), the sign-off STA tool: `make sta STA_TOOL=pt` (the default from `env.sh`) runs `flow/sta/pt_sta.tcl` on the Fusion Compiler netlist of Lab P1 with the compiled library `LIB_DB` and the same SDC. OpenSTA is not installed. The library is the generic `class.db` (Lab P1): its paths are several times slower than sky130's and it has no recovery/removal arcs, so your numbers will not resemble the illustrative ones in this text. The reviewer compares the trends and the reasoning (which path, why, which design wins and by what ratio), not absolute values.

By the end of this lab you should be able to:

1. Define startpoint, endpoint, arrival time, required time, slack, WNS and TNS, and write the setup and hold inequalities.
2. Write an SDC file: clock, uncertainty, input and output delays, driving cells and loads, and explain each number.
3. Run OpenSTA through a generic script and `make sta CLK_PERIOD=<ns>`.
4. Read every column of an OpenSTA path report and map the cells of the critical path to RTL statements.
5. Find the maximum frequency of a block and explain the area/latency/throughput trade-off between two implementations.
6. Explain hold, recovery and removal checks and why they behave as they do with an ideal clock.
7. State what this analysis leaves out (wires, clock tree, corners) and how far its numbers can be trusted.

Where this lab sits. It analyzes the sky130 netlists that `make synth` (Lab P1) writes. The `sta` target and the SDC style are reused by the parameter sweeps of Lab P3, by the CDC constraints of Labs T-RTL1 and T-RTL2, and by Lab T-RTL3, which starts from the critical path you find here and pipelines the multiplier to a frequency target.

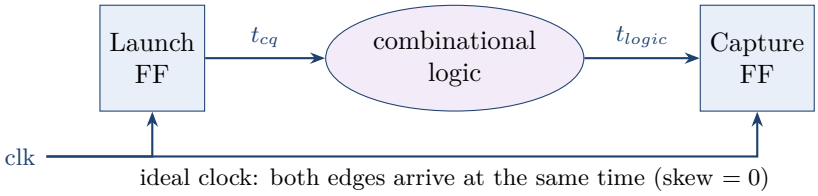
2 Static Timing Analysis Concepts

2.1 Paths, Startpoints and Endpoints

STA cuts the design at its sequential elements and ports. Every timing path starts at a **startpoint** (a flip-flop clock pin or an input port) and ends at an **endpoint** (a flip-flop data pin or an output port). This gives four kinds of paths:

Path	Constrained by	Example in your designs
Register to register	Clock period	Product register → adder → product register (`seq_mul`)
Input to register	Clock period minus input delay	`in_req` → operand load mux → operand register
Register to output	Clock period minus output delay	State register → `in_ready`
Input to output	Period minus both delays	`push` → `full`? (only if combinational: check your FIFO)

2.2 Setup and Hold



With clock period T, clock-to-Q delay t_{cq}, logic delay t_{logic}, clock uncertainty t_{unc} and clock skew t_{skew} (capture clock later than launch clock):

$$\textbf{setup:}\quad t_{cq} + t_{logic} \;\le\; T + t_{skew} - t_{setup} - t_{unc} \qquad \textbf{hold:}\quad t_{cq} + t_{logic} \;\ge\; t_{hold} + t_{skew} + t_{unc}$$

The left side is the **arrival time**, the right side the **required time**, and **slack** = required − arrival for setup (= arrival − required for hold). Positive slack means the check is met. Setup is a check against the *next* clock edge (the data must not be too slow); hold is a check against the *same* edge (the new data must not be so fast that it corrupts the value being captured). Note that T does not appear in the hold inequality: **a hold violation cannot be fixed by slowing the clock**, which is why it is the more dangerous one in silicon.

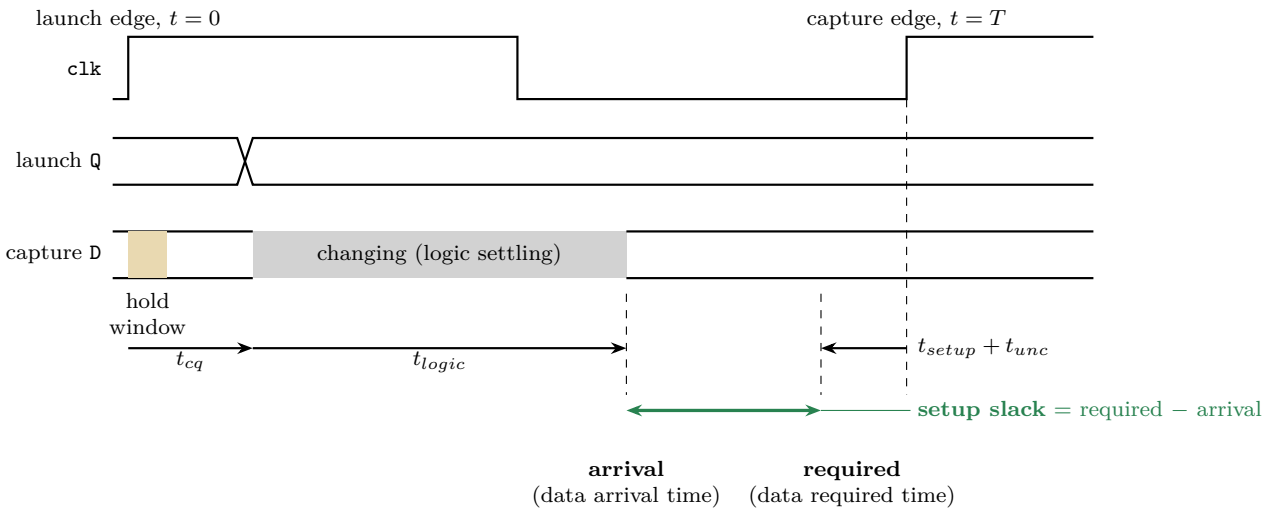


Figure 1: One register-to-register path in time. The launch edge starts the path; the data reaches the capture flip-flop at the *arrival time* $t_{cq} + t_{logic}$; it must be stable a setup time (plus the uncertainty) before the capture edge, the *required time*. Setup slack is the distance between the two. Hold looks at the *same* launch edge: the new data must not reach `D` inside the hold window of the capture flip-flop (here it changes only after t_{cq}, so hold is met).

Figure 1 shows the setup inequality as waveforms: every line of a timing report (Section 5) is one of these intervals.

- **WNS** (worst negative slack): the most negative slack of all endpoints; 0 if all are met. It decides the frequency.
- **TNS** (total negative slack): the sum of the negative slacks of all failing endpoints. It measures how much work is left: one bad path (TNS ≈ WNS) is fixed differently from a whole bus failing.
- **Minimum period** $T_{min} \approx T - \mathrm{WNS}_{setup}$ for register-to-register paths, and $F_{max} = 1/T_{min}$.

2.3 Recovery and Removal

For a flip-flop with asynchronous reset, the *assertion* of reset is asynchronous, but its *release* is not: if `rst_n` rises too close to a clock edge, the flip-flop may go metastable, or some flip-flops leave reset one cycle earlier than others. **Recovery** is the setup-like check (reset release must happen at least t_{rec} before the clock edge) and **removal** the hold-like check (t_{rem} after it). The standard solution, a reset synchronizer (assert asynchronously, release synchronously), is built in Lab T-RTL1.

On the Synopsys cloud: PrimeTime times recovery and removal by default and reports them in the path group `**async_default**` (`report_timing -group **async_default** -delay_type max|min`). The generic `class.lib` has no `recovery/removal` arcs, so with `class.db` those reports say *No Paths* — the check exists only if the library characterizes it. With SAED32 the same report shows, for example, `rst_n` → `count_q_reg_0_/RSTB (DFFARX1_RVT)`, `library recovery time 0.092`.

2.4 What This Analysis Leaves Out

The numbers you get here are useful for *comparing* designs and for finding the critical path. They are not sign-off numbers, because:

- **No wires.** There is no placement, so there are no extracted parasitics (SPEF). Net delay is zero and net load is only the sum of pin capacitances (plus a wire-load model if the library defines

a default one). In 130 nm, wires add a modest but real share of delay; in advanced nodes they dominate.

- **Ideal clock.** The clock reaches every flip-flop at the same time with a fixed slew. A real clock tree has latency and skew; `set_clock_uncertainty` is our placeholder margin.
- **One typical corner, no derating.** Sign-off uses slow corners for setup, fast corners for hold, and on-chip-variation derates.
- **No timing-driven synthesis.** Our Yosys flow optimizes mostly for area and does not upsize cells or buffer high-fanout nets against a target. A timing-driven flow would do better.

On the Synopsys cloud: Fusion Compiler *is* timing-driven: it optimizes against the SDC it reads during synthesis. The flow synthesizes at `SYN_CLK_PERIOD` (default 10 ns) and keeps that netlist for every `make sta CLK_PERIOD=<p>`, so the T_{min} you find is that of one fixed netlist, comparable between designs. `make synth sta SYN_CLK_PERIOD=<p> CLK_PERIOD=<p>` asks Fusion Compiler to close timing at p (sizing, restructuring, buffering): a different netlist, a useful extra experiment — report it separately. The netlist is non-physical (zero wire load, ideal clock) just like the Yosys one. PrimeTime does use the library's wire-load model if it has one (SAED32 does, so its input-pin lines show small non-zero net delays; `class.lib` selects `05x05` … `20x20` by area). Consequence: Fusion Compiler's `compile_logical` times with zero interconnect, PrimeTime with the wire-load model, so on `class.db` PrimeTime is up to 20–25 % slower than the synthesis report for the same netlist. PrimeTime is the sign-off number. Either over-constrain synthesis (`SYN_CLK_PERIOD` about 20 % below the target) or let both tools use one explicit model: `make synth sta SYN_WIRE_LOAD=20x20`.

As a rule of thumb, expect a post-route frequency 15–30 % below what this analysis predicts at the typical corner, and much lower at the slow corner. The last section of Lab P3 shows how the open-source place-and-route flow closes this gap.

2.5 Timing Exceptions

STA assumes every path must complete in one cycle. Two SDC commands override this: `set_false_path` (a path that can never be functionally exercised, or a clock-domain crossing handled by a synchronizer) and `set_multicycle_path` (a path whose result is sampled only every N cycles). Exceptions are powerful and dangerous: STA trusts them blindly, and a wrong exception is a silicon bug that no check in this flow will find. No design in the core needs one; Lab T-RTL1 uses them for CDC.

3 Constraints: the SDC File

Each IP has its constraints in `hw/ip/<ip>/syn/<ip>.sdc`. OpenSTA reads SDC as Tcl, so the file can compute values; the period comes from `make` through `CLK_PERIOD`.

```
# hw/ip/seq_mul/syn/seq_mul.sdc -- constraints for seq_mul and comb_mul (Lab P2)
set period   $::env(CLK_PERIOD)                ;# ns, from make CLK_PERIOD=...
set io_delay [expr {0.2 * $period}]            ;# 20% of the cycle used outside

create_clock -name clk -period $period [get_ports clk]
set_clock_uncertainty 0.10 [get_clocks clk]     ;# margin for jitter and skew
set_clock_transition  0.15 [get_clocks clk]     ;# slew of the ideal clock

set clk_port   [get_ports clk]
set non_clk_in [lsearch -inline -all -not -exact [all_inputs] $clk_port]
set_input_delay  $io_delay -clock clk $non_clk_in
set_output_delay $io_delay -clock clk [all_outputs]
```

```
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin Y $non_clk_in
set_load 0.01 [all_outputs]                        ;# pF, a few cell inputs
set_max_fanout 20 [current_design]
```

Command	Meaning and why this value
`create_clock`	Defines the clock on port `clk` with the period under test. Without it, nothing is timed.
`set_clock_uncertainty`	Subtracted from every setup check (added to hold): reserves margin for jitter and the skew of the future clock tree.
`set_clock_transition`	Slew at the flip-flop clock pins; affects clock-to-Q and setup values from the tables.
`set_input_delay`	When the input changes after the clock edge, i.e. how much of the cycle the driving block uses. 20 % is a common budget for a block whose neighbors are unknown.
`set_output_delay`	How much of the cycle the receiving block needs after our output.
`set_driving_cell`	The inputs are driven by a real cell (an `inv_2`), so input slew depends on the load inside the block.
`set_load`	Capacitance the outputs drive outside the block.
`set_max_fanout`	A design rule: nets above this fanout are reported as violations.

The clock port is removed from the input-delay list; `rst_n` keeps its input delay, which is what gives the recovery and removal checks something to check. `sync_fifo.sdc` is the same file with the comment changed; Lab P3 adds the SDC files of `arb_mem` and `dcache` the same way.

On the Synopsys cloud: The same SDC must be read by OpenSTA, PrimeTime and Fusion Compiler. Two lines of the listing are not portable. (1) PrimeTime and Fusion Compiler return *collections*, not Tcl lists: `lsearch` on `[all_inputs]` does not work there; use `remove_from_collection [all_inputs] [get_ports clk]` (test with `info commands remove_from_collection` to keep one file for all three tools). (2) The driving cell and the load are library-specific (`sky130_fd_sc_hd__inv_2/Y`, 0.01 pF); the repository's SDC files read them from `SDC_DRIVE_CELL`, `SDC_DRIVE_PIN` and `SDC_LOAD`, which `syn.mk` sets per library (`class`: `IV/Z` and 4 units of 0.1 fF; SAED32: `INVX2_RVT/Y`, 4 fF), with the sky130 values as default. Fusion Compiler ignores `set_max_fanout` (warning `CSTR-011`); PrimeTime checks it.

4 The Generic STA Script and `sta.mk`

```
# flow/sta/sta.tcl -- generic OpenSTA script (Lab P2)
# Run by sta.mk as:  sta -no_init -exit sta.tcl
set top $::env(DUT_TOP)
set out $::env(STA_OUT)                ;# prefix of the report files

read_liberty $::env(LIB)
read_verilog $::env(NETLIST)
link_design  $top
read_sdc     $::env(SDC)               ;# the SDC reads CLK_PERIOD itself
set sta_recovery_removal_checks_enabled 1

check_setup -verbose                                        > ${out}_check_setup.rpt
report_checks -path_delay max -group_path_count 5 -digits 3 \
    -fields {slew cap input_pins nets fanout}               > ${out}_setup.rpt
report_checks -path_delay min -group_path_count 5 -digits 3  > ${out}_hold.rpt
report_checks -path_delay max -to [all_registers -async_pins] > ${out}_recovery.rpt
```

```
report_checks -path_delay min -to [all_registers -async_pins] > ${out}_removal.rpt
report_check_types -max_slew -max_capacitance -max_fanout -violators > ${out}_drv.rpt

# one-line summaries on stdout (parsed by sweep.sh in Lab P3)
report_worst_slack -max -digits 3
report_worst_slack -min -digits 3
report_wns -digits 3
report_tns -digits 3
report_clock_min_period
```

```
# flow/mk/sta.mk -- static timing analysis (Lab P2)
# From the IP Makefile: DUT_TOP SDC; from syn.mk: NETLIST LIB
CLK_PERIOD ?= 10.0
STA_DIR    := $(BUILD_DIR)/sta
STA_TAG    := $(DUT_TOP)_p$(CLK_PERIOD)

.PHONY: sta
sta: $(NETLIST)
        @mkdir -p $(STA_DIR)
        DUT_TOP=$(DUT_TOP) NETLIST=$(NETLIST) LIB=$(LIB) SDC=$(abspath $(SDC)) \
        CLK_PERIOD=$(CLK_PERIOD) STA_OUT=$(STA_DIR)/$(STA_TAG) \
          sta -no_init -exit $(REPO_ROOT)/flow/sta/sta.tcl | tee $(STA_DIR)/$(STA_TAG).
    log
```

Reports land in `build/<ip>/<cfg>/sta/<top>_p<period>_*.rpt`, so runs at different periods and for different tops do not overwrite each other. `check_setup` must be empty: an unconstrained endpoint is an endpoint STA silently ignores. `report_clock_min_period` gives a quick F_{max} estimate from register-to-register paths only; use it as a cross-check, not as the answer to question 3.

On the Synopsys cloud: `flow/sta/pt_sta.tcl` writes the same report files with PrimeTime commands: `read_verilog` + `link_design` with `link_path "* $LIB_DB"`; `source` of the SDC; `check_timing -verbose` (the `check_setup` equivalent: no clock, unconstrained endpoints, missing input delays); `report_timing -delay_type max -max_paths 5 -nworst 1 -input_pins -nets -transition_time -capacitance` (setup) and `-delay_type min` (hold); `report_timing -group **async_default**` (recovery, removal); `report_constraint -all_violators -max_transition -max_capacitance -max_fanout` (DRV); `report_qor` (WNS, TNS, number of violating paths, logic levels per path group). One trap: with `-max_paths` larger than 1, PrimeTime reports only *violating* paths unless `-slack_lesser_than` is given — a met design then shows "No constrained paths". PrimeTime has no `report_clock_min_period`; use T − WNS of the register-to-register group. The script also writes `<top>_p<period>_summary.txt` (setup slack, TNS, hold slack, start- and endpoint, path type) for the sweep of Lab P3. Run it with `make sta`; `pt_shell` reads its script from `-f` with `< /dev/null` so that an error never leaves it waiting at the prompt.

5 Anatomy of a Timing Report

The setup report below is for `seq_mul` (`XLEN=32`) at `CLK_PERIOD=10.0`. It is **illustrative**: realistic OpenSTA format and plausible sky130 typical-corner values, consistent with the `stat` example in Lab P1, but your design will give different cells and numbers. The bracketed numbers are annotations, not part of the report.

```
Startpoint: _0835_ (rising edge-triggered flip-flop clocked by clk)         [1]
Endpoint: _0899_ (rising edge-triggered flip-flop clocked by clk)           [1]
Path Group: clk
Path Type: max                                                              [2]
```

```
Fanout     Cap    Slew   Delay    Time   Description                          [3]
---------------------------------------------------------------------------
                  0.000   0.000   0.000   clock clk (rise edge)               [4]
                          0.000   0.000   clock network delay (ideal)         [5]
                  0.150   0.000   0.000 ^ _0835_/CLK (sky130_fd_sc_hd__dfxtp_1)
                  0.062   0.338   0.338 ^ _0835_/Q (sky130_fd_sc_hd__dfxtp_1)   [6]
     2    0.004                           b_q[0] (net)                         [7]
                  0.062   0.000   0.338 ^ _0601_/A (sky130_fd_sc_hd__buf_2)   [8]
                  0.260   0.174   0.512 ^ _0601_/X (sky130_fd_sc_hd__buf_2)
    33    0.071                           _0217_ (net)                         [9]
                  0.260   0.002   0.514 ^ _0688_/B (sky130_fd_sc_hd__nand2_1)
                  0.091   0.118   0.632 v _0688_/Y (sky130_fd_sc_hd__nand2_1)  [10]
     2    0.005                           _0254_ (net)
  ... 19 cells of the 64-bit adder omitted (xnor2, maj3, a21oi, o21ai) ...
                  0.071   0.000   3.064 ^ _0791_/A1 (sky130_fd_sc_hd__mux2_1)
                  0.048   0.338   3.402 ^ _0791_/X (sky130_fd_sc_hd__mux2_1)
     1    0.002                           prod_d[63] (net)
                  0.048   0.000   3.402 ^ _0899_/D (sky130_fd_sc_hd__dfxtp_1)
                                  3.402   data arrival time                    [11]

                         10.000  10.000   clock clk (rise edge)               [12]
                          0.000  10.000   clock network delay (ideal)
                         -0.100   9.900   clock uncertainty                    [13]
                          0.000   9.900   clock reconvergence pessimism
                                  9.900 ^ _0899_/CLK (sky130_fd_sc_hd__dfxtp_1)
                         -0.112   9.788   library setup time                  [14]
                                  9.788   data required time
---------------------------------------------------------------------------
                                  9.788   data required time
                                 -3.402   data arrival time
---------------------------------------------------------------------------
                                  6.386   slack (MET)                         [15]
```

5.1 The Columns

Column	Meaning
Fanout	On net lines: number of pins the net drives
Cap	On net lines: total capacitance on the net in pF (pin capacitances; no wire here)
Slew	Transition time at the pin in ns; it indexes the delay table of the next cell
Delay	Increment of this line: a cell delay on an output pin, a net (wire) delay on an input pin — always 0.000 here because there are no wires
Time	Cumulative arrival time at this point
`^` / `v`	Rising / falling transition at the pin; inverting cells flip it
Description	Pin (`instance/pin`) with cell type, or net name, or a clock event

5.2 The Annotations

[1] Start- and endpoint: instance names of flip-flops. Map them to RTL through the Q net (P1): `_0835_` drives `b_q[0]`, the multiplier LSB; `_0899_` captures `prod_d[63]`, the MSB of the product register.

[2] `max`: a setup check (latest arrival). `min` is hold.

[3] Header of the columns selected with `-fields`.

[4] Launch edge at time 0.

[5] Ideal clock: no tree, so zero latency. After clock-tree synthesis this line holds the real insertion delay.

[6] Clock-to-Q of the launching flip-flop: 0.338 ns, a significant part of the budget at high frequency.

[7] `b_q[0]` decides whether the multiplicand is added: it must reach every bit of the adder input.

[8] Input pins appear because of `-fields {input_pins}`; the delay into them is the (zero) wire delay.

[9] Fanout 33 and 0.071 pF: a heavily loaded net, slow edge (slew 0.260). This is also a `max_fanout` violation in the DRV report: synthesis inserted one buffer but did not fix it.

[10] A falling output (`v`): `nand2` inverts.

[11] Data arrival time: when the last bit of the new product is stable at the D pin.

[12] Capture edge: one period later.

[13] Uncertainty from the SDC, subtracted for setup.

[14] Setup time of `dfxtp_1`, looked up from the liberty table with the actual data and clock slews.

[15] Slack $= 9.788 - 3.402 = 6.386$ ns, met. This path alone would allow $T_{min} \approx 10 - 6.386 = 3.614$ ns (≈ 277 MHz).

5.3 A Hold Report

The same design, `-path_delay min` (illustrative). The shortest path is the shift of the multiplier register through its load mux:

```
Startpoint: _0840_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: _0839_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: min

  Delay    Time   Description
---------------------------------------------------------
  0.000   0.000   clock clk (rise edge)
  0.000   0.000   clock network delay (ideal)
  0.000   0.000 ^ _0840_/CLK (sky130_fd_sc_hd__dfxtp_1)
  0.301   0.301 v _0840_/Q (sky130_fd_sc_hd__dfxtp_1)
  0.287   0.588 v _0612_/X (sky130_fd_sc_hd__mux2_1)
  0.000   0.588 v _0839_/D (sky130_fd_sc_hd__dfxtp_1)
          0.588   data arrival time

  0.000   0.000   clock clk (rise edge)
  0.000   0.000   clock network delay (ideal)
  0.100   0.100   clock uncertainty
  0.000   0.100   clock reconvergence pessimism
          0.100 ^ _0839_/CLK (sky130_fd_sc_hd__dfxtp_1)
 -0.041   0.059   library hold time
          0.059   data required time
---------------------------------------------------------
          0.059   data required time
         -0.588   data arrival time
---------------------------------------------------------
          0.529   slack (MET)
```

Here the capture edge is the *same* edge (time 0), uncertainty is *added* to the requirement, and the hold time is negative (the flip-flop tolerates data changing slightly before the edge).

5.4 The Same Report in PrimeTime

A PrimeTime setup report has the same content in a slightly different layout. Below: `counter` (Lab L0) synthesized with Fusion Compiler to SAED32, at 10 ns, with the options of `pt_sta.tcl` (abridged; a real report, not illustrative).

```
  Startpoint: clr (input port clocked by clk)
  Endpoint: count_q_reg_1_ (rising edge-triggered flip-flop clocked by clk)
  Path Group: clk
  Path Type: max
  Point                     Fanout    Cap      Trans       Incr       Path
```

```
clock clk (rise edge)                          0.150      0.000      0.000
clock network delay (ideal)                               0.000      0.000
input external delay                                      2.000      2.000 r
clr (in)                                       0.007      0.004      2.004 r
clr (net)                       3    2.615
ctmi_151/A (INVX0_RVT)                         0.007      0.017      2.022 r
ctmi_151/Y (INVX0_RVT)                         0.135      0.088      2.110 f
ctmn_137 (net)                 15   13.889
...
count_q_reg_1_/D (DFFARX1_RVT)                 0.036      0.013      2.253 f
data arrival time                                                    2.253
clock clk (rise edge)                          0.150     10.000     10.000
count_q_reg_1_/CLK (DFFARX1_RVT)                                     10.000 r
clock reconvergence pessimism                             0.000     10.000
clock uncertainty                                        -0.100      9.900
library setup time                                       -0.023      9.877
data required time                                                   9.877
slack (MET)                                                          7.624
```

On the Synopsys cloud: Differences from OpenSTA: the columns are `Point`, `Fanout`, `Cap`, `Trans` (= slew), `Incr` (= Delay) and `Path` (= Time), with the point first; the transition is `r`/`f` at the end of the line instead of `^`/`v` in front of the pin; an input port path starts with `input external delay` (the `set_input_delay`); `clock uncertainty` is listed after `clock reconvergence pessimism`; flip-flop names are Fusion Compiler's (`count_q_reg_1_` = bit 1 of `count_q`). Input-pin lines carry a small net delay here because SAED32 has a wire-load model. Annotate *your* PrimeTime report in the same way as above.

6 Timing Analysis of `seq_mul` and `comb_mul`

Answer each question in `hw/ip/seq_mul/doc/ppa.md` with the relevant excerpt of the OpenSTA output.

On the Synopsys cloud: Use the PrimeTime reports instead: WNS and TNS are in `<top>_p<period>_qor.rpt` (`Critical Path Slack`, `Total Negative Slack`) and in the summary file; the hold report answers question 4; question 5 needs the `**async_default**` group (empty with `class.db`, see above: say so and explain what the check would protect against); for question 6 edit the input delay in a copy of the SDC and pass it with `make sta STA_SDC=<copy>`: `STA_SDC` (default `SDC`) changes only the analysis, whereas `SDC=<copy>` would also re-synthesize, because Fusion Compiler synthesizes against `SDC` and the netlist depends on it. The reports of both runs have the same names (one period): rename the first before the second run.

1. **Setup check at 10 ns.** Is timing met for `seq_mul`? What are the worst negative slack (WNS) and total negative slack (TNS)?
2. **Critical path.** Name its startpoint and endpoint. Which RTL statements does it pass through? Map at least three cells in the path to the RTL that created them (P1 technique).
3. **Maximum frequency.** Reduce `CLK_PERIOD` step by step until WNS is approximately zero (bisect: 10, 5, 3.5, ...). Report the minimum period and the resulting maximum frequency. The I/O delays scale with the period in our SDC; say whether the limiting path is register-to-register or involves a port.
4. **Hold check.** Explain what the `-path_delay min` report checks and why hold slack is positive here even though there is no clock tree.
5. **Asynchronous reset.** Find the recovery and removal checks on `rst_n`. What do they protect against?
6. **I/O constraints.** Change the input delay from 2 ns to 6 ns (edit `io_delay` for inputs only, at `CLK_PERIOD=10`). Which paths become critical, and why does the internal logic not change?

7. **Comparison.** Repeat questions 1–3 for `comb_mul` (`make synth sta DUT_TOP=comb_mul`) and fill in the table below.
8. **Improvement.** Make one RTL change to `seq_mul` that shortens its critical path. Measure the result and explain the trade-off (Section 7).

Metric	`seq_mul`	`comb_mul`
Standard cells (count)		
Area (μm^2)		
Flip-flops (count)		
Minimum clock period (ns)		
Latency (cycles)		
Time for one multiplication (ns)		
Multiplications per μs, back-to-back		
Area × time per multiplication (μm^2·ns)		

Finish with one paragraph: when would you choose each design in a processor? Consider how often the core multiplies, whether the multiplier may stall the pipeline, and the clock frequency the rest of the core reaches. For a sanity check of orders of magnitude: in this flow a 32-bit array multiplier typically lands at several times the area of the sequential unit and at a minimum period several times longer; if your numbers differ by much more than that, look for a missing register or an unintended path in your RTL.

7 Improvement Task

Shorten the critical path of `seq_mul` with one RTL change, rerun V1 regression and P1 GLS (the change must not break anything), and measure cells, area and T_{min} again.

Test term — Regression. All tests of `dv/regress.list` over their seeds and parameter sets, one command (`make regress`), one pass/fail summary. A pass after the change proves that the faster RTL still does everything the old one did; that is why a timing improvement is never merged without it.

Test term — Gate-level simulation (GLS). The V1 tests run on the new netlist (`make synth gls`, Lab P1). It proves that the changed RTL still synthesizes to the hardware it simulates (no new mismatch, no missing reset); STA cannot show that, because STA does not know what the logic is supposed to compute.

Candidate changes, from simplest to most involved:

- **Halve the adder.** The classic shift-and-add keeps the product in a 2×XLEN register, adds the multiplicand to the *upper half* only, and shifts the whole register right; the multiplier occupies the lower half and is consumed as the product grows. The adder shrinks from 2×XLEN to XLEN+1 bits and the separate multiplicand shift register disappears: both area and delay drop.
- **Precompute the add decision.** Register `b_q[0]`-dependent control one cycle early, or gate the adder input with a registered select, so that the high-fanout net in the report starts from a flip-flop with its own buffer tree. Watch the effect on fanout and slew in the report.
- **Carry-save accumulation.** Keep the partial product in carry-save form (sum and carry vectors, no carry propagation per iteration) and resolve it with one carry-propagate addition at the end.

The per-cycle path becomes a single full-adder level; the price is a second register and one extra cycle.

The opposite direction is also instructive: a radix-4 step (two bits per cycle) halves the latency but lengthens the critical path. Whatever you choose, report the before/after table (cells, area, flip-flops, T_{min}, latency, time per multiplication) and explain the trade-off in two or three sentences. Lab T-RTL3 goes further: pipelining to a target frequency.

8 `sync_fifo`: Effect of WIDTH and DEPTH

Constrain `sync_fifo` with `hw/ip/sync_fifo/syn/sync_fifo.sdc` and analyze the parameter sets below. Before running, *predict* which quantity grows linearly and which logarithmically.

PARAMS	FFs	Area (μm^2)	T_{min} (ns)	Path type	Critical path (start → end)
`WIDTH=8 DEPTH=16`					
`WIDTH=32 DEPTH=16`					
`WIDTH=8 DEPTH=64`					
`WIDTH=32 DEPTH=5`					

Questions:

1. Storage is `WIDTH`×`DEPTH` flip-flops; what else grows with `DEPTH`? (Read multiplexer: a `DEPTH`:1 mux per data bit, whose depth grows with $\log_2$`DEPTH`; write-enable decoding; pointer width.)
2. Does `WIDTH` change T_{min}? Explain through the structure of the critical path, not through the numbers.
3. With first-word fall-through, `pop_data` comes from the storage through the read mux. Is the critical path register-to-output? How does the output delay budget affect it, and what would a registered output cost?
4. `full` and `empty` come from registers (R2 spec). Where would the critical path be if they were computed combinationally from `count`?
5. `DEPTH=5` is not a power of two. Find the pointer-wrap logic in the netlist and compare its cost with `DEPTH=16`.

9 Requirements / Tasks

1. Write `flow/sta/sta.tcl` and `flow/mk/sta.mk`; include `sta.mk` in the Makefiles of `seq_mul` and `sync_fifo`; set `SDC` in both.
2. Write `hw/ip/seq_mul/syn/seq_mul.sdc` and `hw/ip/sync_fifo/syn/sync_fifo.sdc`; justify every value in a comment. `check_setup` must report no unconstrained endpoints.
3. Annotate your own `seq_mul` setup report at 10 ns the way Section 5 does (every distinct line type explained).
4. Answer the eight timing questions with report excerpts.
5. Fill in the `seq_mul`/`comb_mul` table and write the paragraph.
6. Perform the improvement task with a before/after table. V1 regression and P1 GLS still pass after the change.
7. Fill in the `sync_fifo` table and answer its questions.

8. Review the DRV report (`max_fanout`, `max_slew`, `max_capacitance`) of each design; explain each violation and how a back-end flow would fix it.

10 Repository and Flow

Path	Change in this lab
`flow/sta/sta.tcl`	New: generic OpenSTA script
`flow/mk/sta.mk`	New: `sta` target, `CLK_PERIOD`
`hw/ip/seq_mul/syn/seq_mul.sdc`	New: constraints (used for `seq_mul` and `comb_mul`)
`hw/ip/sync_fifo/syn/sync_fifo.sdc`	New: constraints
`hw/ip/seq_mul/Makefile`, `hw/ip/sync_fifo/Makefile`	`SDC := syn/<ip>.sdc`, `include $(REPO_ROOT)/flow/mk/sta.mk`
`hw/ip/seq_mul/rtl/seq_mul.sv`	Improvement task (one change, own commit)
`hw/ip/<ip>/doc/ppa.md`	Timing section added
`build/<ip>/<cfg>/sta/`	Generated reports and logs

Target / variable	Function
`sta`	Synthesizes if needed, then runs OpenSTA with `SDC` at `CLK_PERIOD`; writes setup, hold, recovery, removal, DRV and check reports
`CLK_PERIOD`	Clock period in ns (default 10.0); read by the SDC
`SDC`	Per-IP constraints file, set in the IP Makefile

On the Synopsys cloud: PrimeTime path: `flow/sta/pt_sta.tcl` next to `sta.tcl`; `sta.mk` dispatches on `STA_TOOL` (`opensta|pt`) and uses `LIB_DB` instead of `LIB`. Additional outputs: `<top>_p<period>_qor.rpt` and `<top>_p<period>_summary.txt`; the PrimeTime log is `<top>_p<period>.log` (its first line, `Error: Library Compiler executable path is not set (PT-063)`, is harmless: it only means `lc_shell` is not in `PATH`). `SDC_DRIVE_CELL`, `SDC_DRIVE_PIN`, `SDC_LOAD`: library values read by the SDC.

```
cd $REPO_ROOT/hw/ip/seq_mul
make sta                                    # default 10 ns
make sta CLK_PERIOD=4.0
make synth sta DUT_TOP=comb_mul CLK_PERIOD=12.0
less $REPO_ROOT/build/seq_mul/default/sta/seq_mul_p4.0_setup.rpt
cd ../sync_fifo
make sta PARAMS="WIDTH=8 DEPTH=64" CLK_PERIOD=5.0
```

On the Synopsys cloud: The same commands work unchanged with `STA_TOOL=pt`. With `class.db` start the bisection higher (`CLK_PERIOD=20`): the 32-bit designs need more than 10 ns in that library.

11 Deliverables

Merge request `lab/p2-sta` (tag `p2-done`) containing:

- `flow/sta/sta.tcl`, `flow/mk/sta.mk`, both SDC files, updated IP Makefiles.
- The improvement change to `seq_mul.sv` as a separate commit, with the regression and GLS results in its message.

- `hw/ip/seq_mul/doc/ppa.md`, timing section: your annotated report, answers to the eight questions with excerpts, the comparison table and paragraph, the improvement before/after table, DRV discussion.
- `hw/ip/sync_fifo/doc/ppa.md`, timing section: the parameter table, predictions versus results, answers.
- README of both IPs: the `sta` command and the headline numbers (T_{min}, area) for the default configuration.

12 Assessment

Criterion	Weight
Constraints: SDC correct, every value justified, no unconstrained endpoints	15%
Generic flow: `sta.tcl`, `sta.mk`, reproducible reports	10%
Report reading: annotated report, path mapped to RTL	15%
The eight timing questions	25%
`seq_mul`/`comb_mul` comparison and paragraph	10%
Improvement task: measured, verified, trade-off explained	15%
`sync_fifo` parameter study	5%
Documentation (`ppa.md`, README)	5%
Total	100%

13 Review Questions

1. Which line of the timing report tells you the design meets timing, and which RTL statement limits the frequency?
2. Write the setup and hold inequalities. Why can a hold violation not be fixed by lowering the clock frequency?
3. What is the difference between WNS and TNS, and what does a large TNS with a small WNS tell you?
4. Why does the `Delay` column show 0.000 on every input pin, and what will change after place and route?
5. What does `set_input_delay 2.0` say about the world outside the block? What happens if it is too optimistic?
6. Why is `clk` excluded from `set_input_delay`, and why is `rst_n` not?
7. A net has fanout 33. Why is the path through it slow, given that the driving cell is the same as elsewhere?
8. Why might the post-route F_{max} of `seq_mul` be lower than your result, and which SDC line partly anticipates it?
9. `comb_mul` produces one result per cycle and `seq_mul` one per XLEN cycles. Under which conditions is `seq_mul` still the better choice?
10. When is `set_multicycle_path` legitimate, and why is a wrong one dangerous?

14 References

- OpenSTA repository and `doc/OpenSTA.pdf`: commands `report_checks`, `report_check_types`, `check_setup`, `report_clock_min_period`.
- Synopsys, *Using the Synopsys Design Constraints Format* (SDC application note), and the open SDC specification distributed with OpenSTA.
- J. Bhasker and R. Chadha, *Static Timing Analysis for Nanometer Designs: A Practical Approach*, Springer: chapters on STA concepts, timing checks and timing exceptions.
- Weste and Harris, *CMOS VLSI Design*: chapters on sequential circuit timing and delay models.
- SkyWater SKY130 PDK documentation: `sky130_fd_sc_hd` timing characterization and corners.
- RISC-V unprivileged specification, M extension (context for the multiplier comparison).